You

I'm currently using docx in python to extract data from a form in Microsoft Word. How can I put the data into a new row in an excel table without overwriting the original and into the correct columns?

Copilot

That's a great project you're working on! To achieve this, you can use the openpyxl library in Python to interact with your Excel file. First, you'll extract the data from the Word document using python-docx, and then you'll write that data into a new row in your Excel sheet. Here's a basic example to get you started:

1. **Install the necessary libraries** (if you haven't already):
   pip install python-docx openpyxl
2. **Extract data from the Word document**:

```
from docx import Document

# Load the Word document
doc = Document('your_form.docx')

# Assuming your form data is in table format, you can extract it like this
data = []
for table in doc.tables:
    for row in table.rows:
        row_data = []
        for cell in row.cells:
            row_data.append(cell.text)
        data.append(row_data)

# Process your data as needed (e.g., extract specific columns or format it)
# Here I'll just keep it simple and use the extracted data directly
```

3. **Write data to an Excel file**:

```
from openpyxl import load_workbook

# Load the Excel workbook and select the desired sheet
wb = load_workbook('your_excel_file.xlsx')
ws = wb['Sheet1']  # Replace 'Sheet1' with the name of your sheet

# Append data to the next empty row
for row_data in data:
    ws.append(row_data)

# Save the updated Excel file
wb.save('your_excel_file.xlsx')
```

This code will append the extracted data from the Word document to the next empty row in the specified Excel sheet. Make sure to adjust the data processing part to fit the structure of your form and the desired columns in your Excel file.

If you need further customization or run into any issues, feel free to ask!

You

For each of these attributes, can you generate a dictionary with the question that will be asked to the user and an empty string for the answer (to be filled later):
[Attributes list redacted]
For example for name the dictionary should hold "What is the name of young person?" for the key and "" for the value

Copilot

Sure, here's a dictionary with the questions and empty strings for the answers:

```
questions = {
    "What is the name of the young person?": "",
    "What is the address of the young person?": "",
    "What is the postcode of the young person?": "",
    "What is the date of birth (DOB) of the young person?": "",
    "What is the age of the young person at referral?": "",
    "What is the gender of the young person?": "",
    "Is the young person involved in or at risk of participation in Serious Organised and Violent Crime?": "",
    "Is the young person involved in or at risk of participation in urban street gang activity?": "",
    "Is the young person involved in or at risk of participation in County Lines?": "",
    "Is the young person at risk of being criminally groomed, or may be grooming others?": "",
    "Is the young person actively involved in Anti-Social Behaviour and has had a yellow or higher ASB Sanction?": "",
    "Is the young person having frequent or out of character Missing Incidents?": "",
    "Has the young person had A&E attendance for injury attributable to violence?": "",
    "What is the name of the young person's school?": "",
    "Who is the referrer?": "",
    "What is the referral date?": ""
}
```

You can populate the empty strings with the actual answers later. Let me know if you need any further customization or help with your code!

You

How can I replace a specific row in the spreadsheet while keeping formatting (e.g. cell borders, bold text, etc.)?

Copilot

To replace a specific row in an Excel spreadsheet while preserving the formatting (such as cell borders, bold text, etc.), you can use the openpyxl library in Python. Here's an example that shows you how to do this:

1. **Install the necessary library** (if you haven't already):

   ```
   pip install openpyxl
   ```

2. **Replace a specific row while preserving formatting**:

```
from openpyxl import load_workbook
from openpyxl.styles import Font, Border, Side

def copy_formatting(source_cell, target_cell):
    target_cell.font = source_cell.font
    target_cell.border = source_cell.border
    target_cell.fill = source_cell.fill
    target_cell.number_format = source_cell.number_format
    target_cell.protection = source_cell.protection
    target_cell.alignment = source_cell.alignment

# Load the Excel workbook and select the desired sheet
wb = load_workbook('your_excel_file.xlsx')
ws = wb['Sheet1']  # Replace 'Sheet1' with the name of your sheet

# Data to replace in the specific row
new_data = ["New Name", "New Address", "New Postcode", "New DOB", "New Age", "New Gender",
            "New Info 1", "New Info 2", "New Info 3", "New Info 4", "New Info 5", "New Info 6",
            "New Info 7", "New School", "New Referrer", "New Referral Date"]

# Specify the row you want to replace (e.g., row 2)
row_to_replace = 2

# Replace data in the specified row while preserving formatting
```

```
for col, new_value in enumerate(new_data, start=1):
    cell = ws.cell(row=row_to_replace, column=col)
    new_cell = ws.cell(row=row_to_replace, column=col, value=new_value)
    copy_formatting(cell, new_cell)

# Save the updated Excel file
wb.save('your_excel_file.xlsx')
```

This script uses the *copy_formatting* function to copy the formatting from the original cell to the new cell. The *new_data* list contains the values you want to replace in the specified row. Adjust the *row_to_replace* and *new_data* according to your requirements.

Let me know if you need any further customization or assistance with your code!

You

I get the following error:

```
Traceback (most recent call last):
  File "/workspace/BCFT-AI/Other AI Projects/AI Document Extraction/doc/blank/pyxlTest.py", line 70, in <module>
    copy_formatting(cell, new_cell)
  File "/workspace/BCFT-AI/Other AI Projects/AI Document Extraction/doc/blank/pyxlTest.py", line 56, in copy_formatting
    target_cell.font = source_cell.font
    ^^^^^^^^^^^^^^^^
  File "/workspace/.pyenv_mirror/user/current/lib/python3.12/site-packages/openpyxl/styles/styleable.py", line 26, in __set__
    setattr(instance._style, self.key, coll.add(value))
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/workspace/.pyenv_mirror/user/current/lib/python3.12/site-packages/openpyxl/utils/indexed_list.py", line 48, in add
    self.append(value)
  File "/workspace/.pyenv_mirror/user/current/lib/python3.12/site-packages/openpyxl/utils/indexed_list.py", line 43, in append
    if value not in self._dict:
       ^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unhashable type: 'StyleProxy'
```

Copilot

Ah, it looks like you're running into an issue with the *StyleProxy* object in *openpyxl*. This error usually occurs because the *source_cell.font* is being returned as a *StyleProxy* instead of the expected *Font* object.

Here's an updated version of the *copy_formatting* function to handle this issue:

```
from openpyxl import load_workbook
from openpyxl.styles import Font, Border, Side, PatternFill, Alignment, Protection

def copy_formatting(source_cell, target_cell):
    if source_cell.has_style:
        target_cell.font = Font(bold=source_cell.font.bold, italic=source_cell.font.italic,
                                name=source_cell.font.name, size=source_cell.font.size,
                                underline=source_cell.font.underline, strike=source_cell.font.strike,
                                color=source_cell.font.color)
        target_cell.border = Border(left=source_cell.border.left, right=source_cell.border.right,
                                    top=source_cell.border.top, bottom=source_cell.border.bottom)
        target_cell.fill = PatternFill(fill_type=source_cell.fill.fill_type, start_color=source_cell.fill.start_color,
                                       end_color=source_cell.fill.end_color)
        target_cell.number_format = source_cell.number_format
        target_cell.protection = Protection(locked=source_cell.protection.locked, hidden=source_cell.protection.hidden)
```

```
        target_cell.alignment = Alignment(horizontal=source_cell.alignment.horizontal,
vertical=source_cell.alignment.vertical,
                                          wrap_text=source_cell.alignment.wrap_text)

# Load the Excel workbook and select the desired sheet
wb = load_workbook('your_excel_file.xlsx')
ws = wb['Sheet1']  # Replace 'Sheet1' with the name of your sheet

# Data to replace in the specific row
new_data = ["New Name", "New Address", "New Postcode", "New DOB", "New Age", "New Gender",
            "New Info 1", "New Info 2", "New Info 3", "New Info 4", "New Info 5", "New Info 6",
            "New Info 7", "New School", "New Referrer", "New Referral Date"]

# Specify the row you want to replace (e.g., row 2)
row_to_replace = 2

# Replace data in the specified row while preserving formatting
for col, new_value in enumerate(new_data, start=1):
    cell = ws.cell(row=row_to_replace, column=col)
    new_cell = ws.cell(row=row_to_replace, column=col, value=new_value)
    copy_formatting(cell, new_cell)

# Save the updated Excel file
wb.save('your_excel_file.xlsx')
```

This function explicitly copies each style property from the source cell to the target cell, handling each aspect of the cell's formatting individually. This should resolve the *StyleProxy* error you're encountering.

Give this a try and let me know how it works!

You

How can I save an uploaded excel file as a new file at the filepath similar to how I did for Word Docs:

```
def upload_file(self, source_path, destination_path):
    # Upload the file to the localFiles directory
    try:
        # Read the source Word file
        doc = Document(source_path)
        # Save the document to the destination path
        doc.save(destination_path)
        print(f"File successfully uploaded to {destination_path}")
        thisAutoLogger.addToLogs(f"File successfully uploaded to {destination_path}")
        return False
    except FileNotFoundError:
        print("The source file does not exist. Please check the path.")
        thisAutoLogger.addToErrorLogs("The source file does not exist. Please check the path.")
        return True
    except Exception as e:
        print(f"An error occurred: {e}")
        thisAutoLogger.addToErrorLogs(f"An error occurred: {e}")
        return True
```

Copilot

You can adapt your existing function to handle Excel files using the *openpyxl* library. Below is an updated version of your *upload_file* function to save an uploaded Excel file to a new file path while maintaining similar functionality:

```
from openpyxl import load_workbook
import shutil

def upload_excel_file(self, source_path, destination_path):
    try:
```

```
        # Open the source Excel file
        wb = load_workbook(source_path)

        # Save the workbook to the destination path
        wb.save(destination_path)

        print(f"File successfully uploaded to {destination_path}")
        thisAutoLogger.addToLogs(f"File successfully uploaded to {destination_path}")
        return False
    except FileNotFoundError:
        print("The source file does not exist. Please check the path.")
        thisAutoLogger.addToErrorLogs("The source file does not exist. Please check the path.")
        return True
    except Exception as e:
        print(f"An error occurred: {e}")
        thisAutoLogger.addToErrorLogs(f"An error occurred: {e}")
        return True
```

In this function, I've used the *load_workbook* method from *openpyxl* to open the Excel file and then save it to the destination path using the save method. This ensures that the Excel file is copied to the new location while preserving its content and formatting.

Feel free to adapt this function further based on your specific requirements!

Conversation by Fifa Utuber with Copilot on Tuesday, Jan 21 2025